Jessica Lopez PDAT 615G Spring 2026 Module 5 Assignment

I enjoyed this assignment in creating a neural network architect and I learned a lot. For the dataset, I selected the Mulit-class Strawberry Ripeness detection dataset. I started with 10 epochs and ended going with 25 epochs based on the decrease of the loss function and increase in accuracy. For the training, I used the Adam optimizer. From the testing, you will see the training results show a steady improvement in performance, with training accuracy reaching almost 70% and validation accuracy stabilizing around 60–65%. This was showing me the model successfully learned meaningful patterns in the data, though some fluctuations in validation accuracy suggest mild overfitting or sensitivity to the dataset. From the customizations, the use of residual connections helped maintain stable training, while dilated convolutions allowed the model to capture broader contextual information. This is useful where global structure matters.

In customizing the architecture, I do believe this architecture was efficient to extract features from images while keeping the number of parameters manageable. But at the same time, a limitation is that custom architectures often require careful tuning and may not generalize as well as pretrained models without additional optimization or more data.

I do believe for real-world computer vision applications, this type of architecture could be useful for tasks such as other types of agricultural monitoring, another way can be quality inspection in manufacturing, or medical image analysis. This is because these types are looking for both fine texture details and broader contextual patterns for accurate classification.

In [ ]:
```
import os
from PIL import Image
from sklearn.model_selection import train_test_split
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms

# ---------------------------------------------------
# 1. Custom Dataset
# ---------------------------------------------------
class StrawberryRipenessDataset(Dataset):
    def __init__(self, image_dir, label_dir, image_files, transform=None):
        self.image_dir = image_dir
        self.label_dir = label_dir
        self.image_files = image_files
        self.transform = transform

    def __len__(self):
        return len(self.image_files)

    def __getitem__(self, idx):
```

```
        image_name = self.image_files[idx]
        image_path = os.path.join(self.image_dir, image_name)

        # Match label filename to image filename
        base_name = os.path.splitext(image_name)[0]
        label_path = os.path.join(self.label_dir, base_name + ".txt")

        image = Image.open(image_path).convert("RGB")

        # Read label
        with open(label_path, "r") as f:
            first_line = f.readline().strip()

        # Supports:
        # 1) plain class label, e.g. "2"
        # 2) YOLO-style line, e.g. "2 0.52 0.44 0.15 0.18"
        label = int(first_line.split()[0])

        if self.transform:
            image = self.transform(image)

        return image, label


# ---------------------------------------------------
# 2. Transformations
# ---------------------------------------------------
train_transform = transforms.Compose([
    transforms.Resize((150, 150)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(15),
    transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    )
])

test_transform = transforms.Compose([
    transforms.Resize((150, 150)),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485, 0.456, 0.406],
        std=[0.229, 0.224, 0.225]
    )
])

# ---------------------------------------------------
# 3. Paths
# ---------------------------------------------------
train_dir = "all/train"
test_dir = "all/test"
label_dir = "all/labels"

# ---------------------------------------------------
```

```
# 4. Collect image filenames
# ----------------------------------------------------
train_images = [f for f in os.listdir(train_dir) if f.lower().endswith(".jpg
test_images = [f for f in os.listdir(test_dir) if f.lower().endswith(".jpg")

print("Number of training images:", len(train_images))
print("Number of test images:", len(test_images))

# ----------------------------------------------------
# 5. Split training into train/validation
# ----------------------------------------------------
train_files, val_files = train_test_split(
    train_images,
    test_size=0.2,
    random_state=42,
    shuffle=True
)

print("Train split:", len(train_files))
print("Validation split:", len(val_files))

# ----------------------------------------------------
# 6. Create datasets
# ----------------------------------------------------
train_dataset = StrawberryRipenessDataset(
    image_dir=train_dir,
    label_dir=label_dir,
    image_files=train_files,
    transform=train_transform
)

val_dataset = StrawberryRipenessDataset(
    image_dir=train_dir,
    label_dir=label_dir,
    image_files=val_files,
    transform=test_transform
)

test_dataset = StrawberryRipenessDataset(
    image_dir=test_dir,
    label_dir=label_dir,
    image_files=test_images,
    transform=test_transform
)

# ----------------------------------------------------
# 7. Create dataloaders
# ----------------------------------------------------
train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=32, shuffle=False)
test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False)

# ----------------------------------------------------
# 8. Custom Model
# ----------------------------------------------------
import torch.nn.functional as F
```

```
class DepthwiseSeparableConv(nn.Module):
    def __init__(self, in_channels, out_channels, stride=1, dilation=1):
        super().__init__()
        self.depthwise = nn.Conv2d(
            in_channels, in_channels,
            kernel_size=3,
            stride=stride,
            padding=dilation,
            dilation=dilation,
            groups=in_channels,
            bias=False
        )
        self.pointwise = nn.Conv2d(
            in_channels, out_channels,
            kernel_size=1,
            bias=False
        )
        self.bn = nn.BatchNorm2d(out_channels)

    def forward(self, x):
        x = self.depthwise(x)
        x = self.pointwise(x)
        x = self.bn(x)
        return F.relu(x)

class GroupedConvBlock(nn.Module):
    def __init__(self, in_channels, out_channels, groups=2, stride=1):
        super().__init__()
        self.conv = nn.Conv2d(
            in_channels, out_channels,
            kernel_size=3,
            stride=stride,
            padding=1,
            groups=groups,
            bias=False
        )
        self.bn = nn.BatchNorm2d(out_channels)

    def forward(self, x):
        x = self.conv(x)
        x = self.bn(x)
        return F.relu(x)

class CustomResidualBlock(nn.Module):
    def __init__(self, in_channels, out_channels, stride=1, dilation=1, grou
        super().__init__()
        self.conv1 = DepthwiseSeparableConv(
            in_channels, out_channels, stride=stride, dilation=dilation
        )
        self.conv2 = GroupedConvBlock(
            out_channels, out_channels, groups=groups, stride=1
        )

        if stride != 1 or in_channels != out_channels:
            self.shortcut = nn.Sequential(
```

```
                nn.Conv2d(in_channels, out_channels, kernel_size=1, stride=s
                nn.BatchNorm2d(out_channels)
            )
        else:
            self.shortcut = nn.Identity()

    def forward(self, x):
        identity = self.shortcut(x)
        out = self.conv1(x)
        out = self.conv2(out)
        out = out + identity
        return F.relu(out)

class CustomImageNet(nn.Module):
    def __init__(self, num_classes=3):
        super().__init__()

        self.stem = nn.Sequential(
            nn.Conv2d(3, 32, kernel_size=3, stride=2, padding=1, bias=False)
            nn.BatchNorm2d(32),
            nn.ReLU()
        )

        self.block1 = CustomResidualBlock(32, 64, stride=1, dilation=1, grou
        self.block2 = CustomResidualBlock(64, 128, stride=2, dilation=1, gro
        self.block3 = CustomResidualBlock(128, 256, stride=2, dilation=2, gr
        self.block4 = CustomResidualBlock(256, 256, stride=1, dilation=2, gr

        self.global_pool = nn.AdaptiveAvgPool2d((1, 1))
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(256, 128),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(128, num_classes)
        )

    def forward(self, x):
        x = self.stem(x)
        x = self.block1(x)
        x = self.block2(x)
        x = self.block3(x)
        x = self.block4(x)
        x = self.global_pool(x)
        x = self.classifier(x)
        return x

# ----------------------------------------------------
# 9. Set number of classes
# ----------------------------------------------------
num_classes = 3   # change this if your dataset has a different number of ri

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = CustomImageNet(num_classes=num_classes).to(device)

criterion = nn.CrossEntropyLoss()
```

```python
optimizer = optim.Adam(model.parameters(), lr=0.001)

# ---------------------------------------------------
# 10. Training
# ---------------------------------------------------
num_epochs = 5

train_losses = []
train_accuracies = []
val_accuracies = []

for epoch in range(num_epochs):
    model.train()
    running_loss = 0.0
    correct_train = 0
    total_train = 0

    for images, labels in train_loader:
        images = images.to(device)
        labels = labels.to(device)

        optimizer.zero_grad()

        outputs = model(images)
        loss = criterion(outputs, labels)

        loss.backward()
        optimizer.step()

        running_loss += loss.item()

        _, predicted = torch.max(outputs, 1)
        total_train += labels.size(0)
        correct_train += (predicted == labels).sum().item()

    epoch_loss = running_loss / len(train_loader)
    epoch_train_acc = 100 * correct_train / total_train

    train_losses.append(epoch_loss)
    train_accuracies.append(epoch_train_acc)

    # Validation
    model.eval()
    correct_val = 0
    total_val = 0

    with torch.no_grad():
        for images, labels in val_loader:
            images = images.to(device)
            labels = labels.to(device)

            outputs = model(images)
            _, predicted = torch.max(outputs, 1)

            total_val += labels.size(0)
            correct_val += (predicted == labels).sum().item()
```

```
    epoch_val_acc = 100 * correct_val / total_val
    val_accuracies.append(epoch_val_acc)

    print(f"Epoch [{epoch+1}/{num_epochs}] "
          f"Loss: {epoch_loss:.4f} | "
          f"Train Accuracy: {epoch_train_acc:.2f}% | "
          f"Validation Accuracy: {epoch_val_acc:.2f}%")

# ---------------------------------------------------
# 11. Final test evaluation
# ---------------------------------------------------
model.eval()
correct_test = 0
total_test = 0

with torch.no_grad():
    for images, labels in test_loader:
        images = images.to(device)
        labels = labels.to(device)

        outputs = model(images)
        _, predicted = torch.max(outputs, 1)

        total_test += labels.size(0)
        correct_test += (predicted == labels).sum().item()

final_test_accuracy = 100 * correct_test / total_test
print(f"\nFinal Test Accuracy: {final_test_accuracy:.2f}%")

# Plot loss and accuracy
import matplotlib.pyplot as plt

plt.figure(figsize=(8, 5))
plt.plot(range(1, num_epochs + 1), train_losses, marker='o')
plt.title("Training Loss per Epoch")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.grid(True)
plt.show()

plt.figure(figsize=(8, 5))
plt.plot(range(1, num_epochs + 1), train_accuracies, marker='o', label='Trai
plt.plot(range(1, num_epochs + 1), val_accuracies, marker='o', label='Valida
plt.title("Training and Validation Accuracy per Epoch")
plt.xlabel("Epoch")
plt.ylabel("Accuracy (%)")
plt.legend()
plt.grid(True)
plt.show()

# Confustion Matrix
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

all_labels = []
all_preds = []
```

```python
model.eval()
with torch.no_grad():
    for images, labels in test_loader:
        images = images.to(device)
        labels = labels.to(device)

        outputs = model(images)
        _, predicted = torch.max(outputs, 1)

        all_labels.extend(labels.cpu().numpy())
        all_preds.extend(predicted.cpu().numpy())

cm = confusion_matrix(all_labels, all_preds)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot(cmap="Blues", xticks_rotation=45)
plt.title("Confusion Matrix")
plt.show()
```

10 Epoch:

Jessica_Lopez_PDAT_615G_M5.ipynb

Users > jessicalopez > Desktop > PDAT_615G > Jessica_Lopez_PDAT_615G_M5.ipynb > empty cell

Generate + Code + Markdown | Run All Restart Clear All Outputs | Jupyter Variables Outline ...

[20] ✓ 3m 41.0s

```
Number of training images: 566
Number of test images: 566
Train split: 452
Validation split: 114
Epoch [1/10] Loss: 1.1016 | Train Accuracy: 42.48% | Validation Accuracy: 29.82%
Epoch [2/10] Loss: 1.0402 | Train Accuracy: 49.78% | Validation Accuracy: 46.49%
Epoch [3/10] Loss: 1.0104 | Train Accuracy: 49.56% | Validation Accuracy: 50.88%
Epoch [4/10] Loss: 1.0280 | Train Accuracy: 53.32% | Validation Accuracy: 52.63%
Epoch [5/10] Loss: 1.0235 | Train Accuracy: 51.55% | Validation Accuracy: 47.37%
Epoch [6/10] Loss: 1.0042 | Train Accuracy: 55.31% | Validation Accuracy: 51.75%
Epoch [7/10] Loss: 0.9582 | Train Accuracy: 56.86% | Validation Accuracy: 51.75%
Epoch [8/10] Loss: 0.9771 | Train Accuracy: 53.76% | Validation Accuracy: 50.00%
Epoch [9/10] Loss: 0.9547 | Train Accuracy: 57.74% | Validation Accuracy: 52.63%
Epoch [10/10] Loss: 0.9666 | Train Accuracy: 58.41% | Validation Accuracy: 57.02%

Final Test Accuracy: 58.66%
```

15 Epoch:

Jessica_Lopez_PDAT_615G_M5.ipynb

Users > jessicalopez > Desktop > PDAT_615G > Jessica_Lopez_PDAT_615G_M5.ipynb > import os

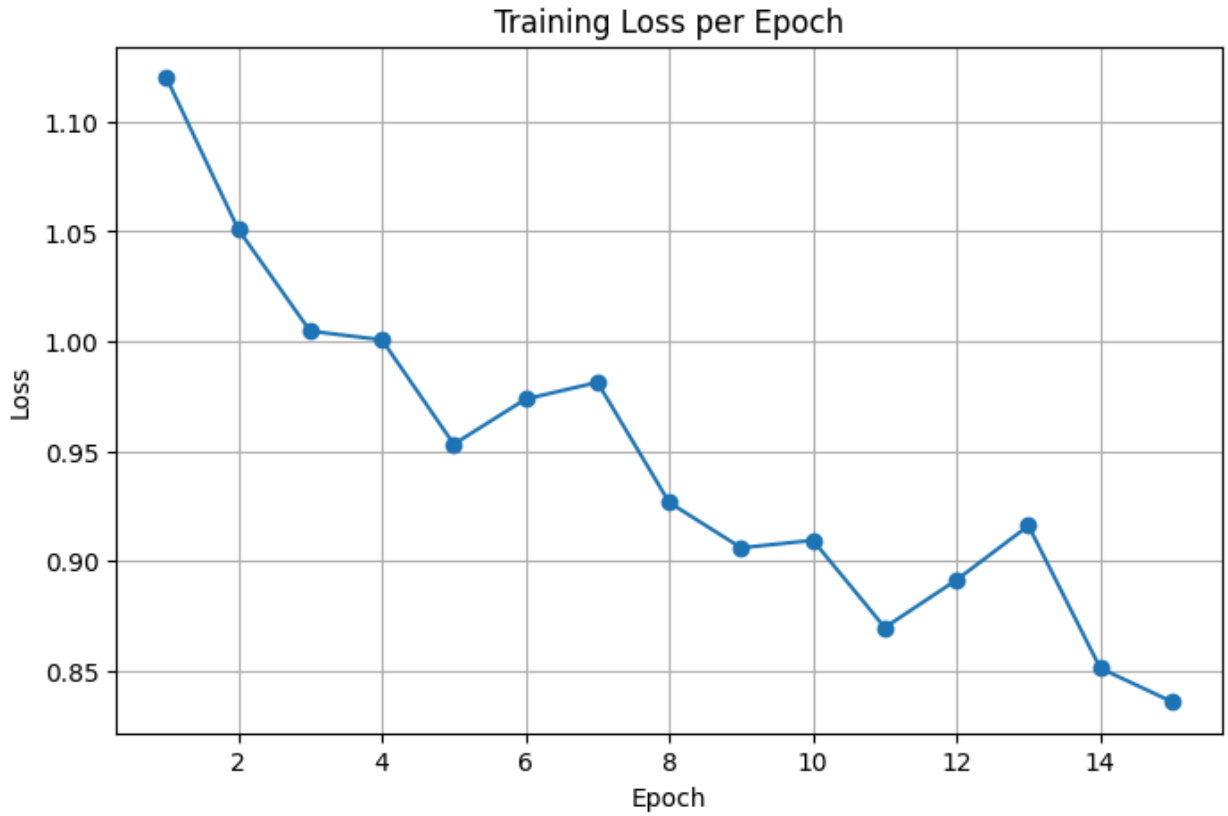
Generate + Code + Markdown | Run All Restart Clear All Outputs | Jupyter Variables Outline ...

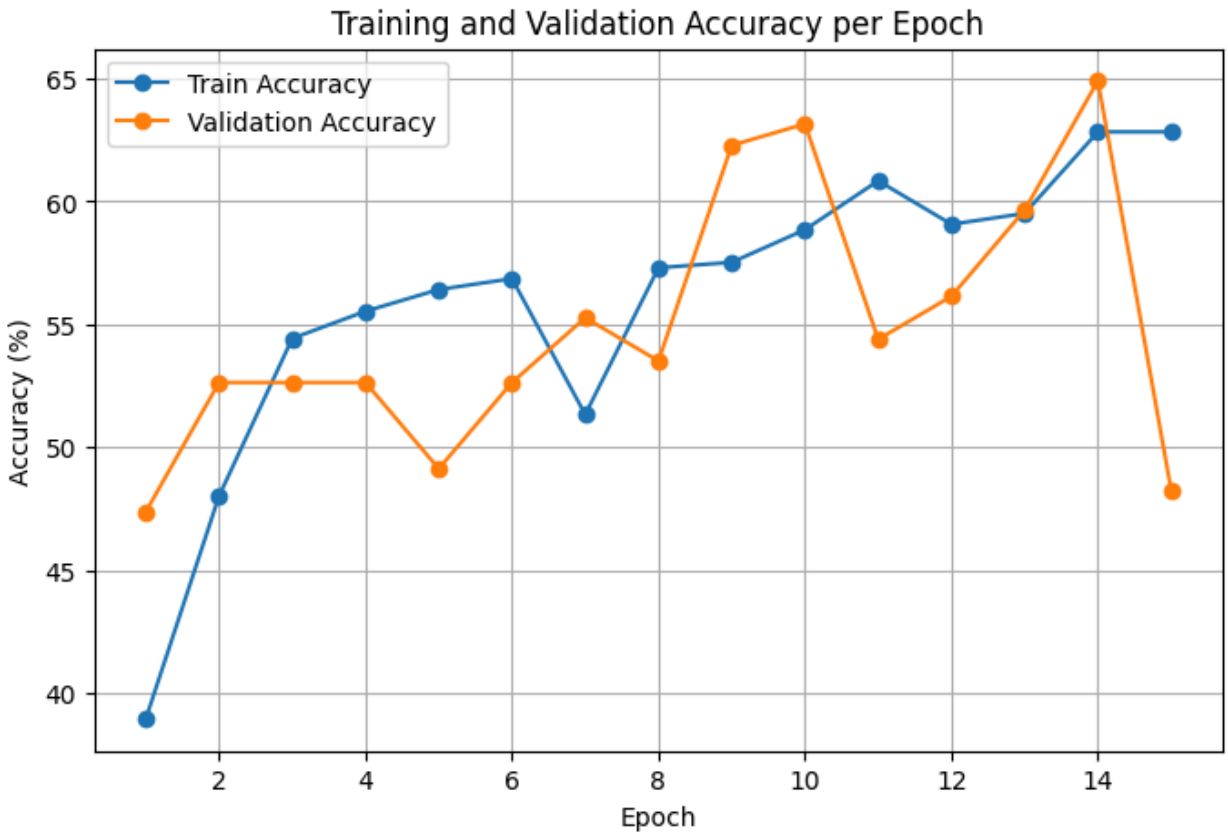
```
plt.show()
```

[22] ✓ 5m 35.8s

```
Number of training images: 566
Number of test images: 566
Train split: 452
Validation split: 114
Epoch [1/15] Loss: 1.1200 | Train Accuracy: 38.94% | Validation Accuracy: 47.37%
Epoch [2/15] Loss: 1.0509 | Train Accuracy: 48.01% | Validation Accuracy: 52.63%
Epoch [3/15] Loss: 1.0046 | Train Accuracy: 54.42% | Validation Accuracy: 52.63%
Epoch [4/15] Loss: 1.0007 | Train Accuracy: 55.53% | Validation Accuracy: 52.63%
Epoch [5/15] Loss: 0.9531 | Train Accuracy: 56.42% | Validation Accuracy: 49.12%
Epoch [6/15] Loss: 0.9737 | Train Accuracy: 56.86% | Validation Accuracy: 52.63%
Epoch [7/15] Loss: 0.9813 | Train Accuracy: 51.33% | Validation Accuracy: 55.26%
Epoch [8/15] Loss: 0.9266 | Train Accuracy: 57.30% | Validation Accuracy: 53.51%
Epoch [9/15] Loss: 0.9060 | Train Accuracy: 57.52% | Validation Accuracy: 62.28%
Epoch [10/15] Loss: 0.9094 | Train Accuracy: 58.85% | Validation Accuracy: 63.16%
Epoch [11/15] Loss: 0.8697 | Train Accuracy: 60.84% | Validation Accuracy: 54.39%
Epoch [12/15] Loss: 0.8914 | Train Accuracy: 59.07% | Validation Accuracy: 56.14%
Epoch [13/15] Loss: 0.9161 | Train Accuracy: 59.51% | Validation Accuracy: 59.65%
Epoch [14/15] Loss: 0.8512 | Train Accuracy: 62.83% | Validation Accuracy: 64.91%
Epoch [15/15] Loss: 0.8359 | Train Accuracy: 62.83% | Validation Accuracy: 48.25%

Final Test Accuracy: 51.77%
```


Training Loss per Epoch
Loss
Epoch
1.10
1.05
1.00
0.95
0.90
0.85
2
4
6
8
10
12
14


Training and Validation Accuracy per Epoch
Train Accuracy
Validation Accuracy
Accuracy (%)
Epoch
65
60
55
50
45
40
2
4
6
8
10
12
14

25 Epoch:

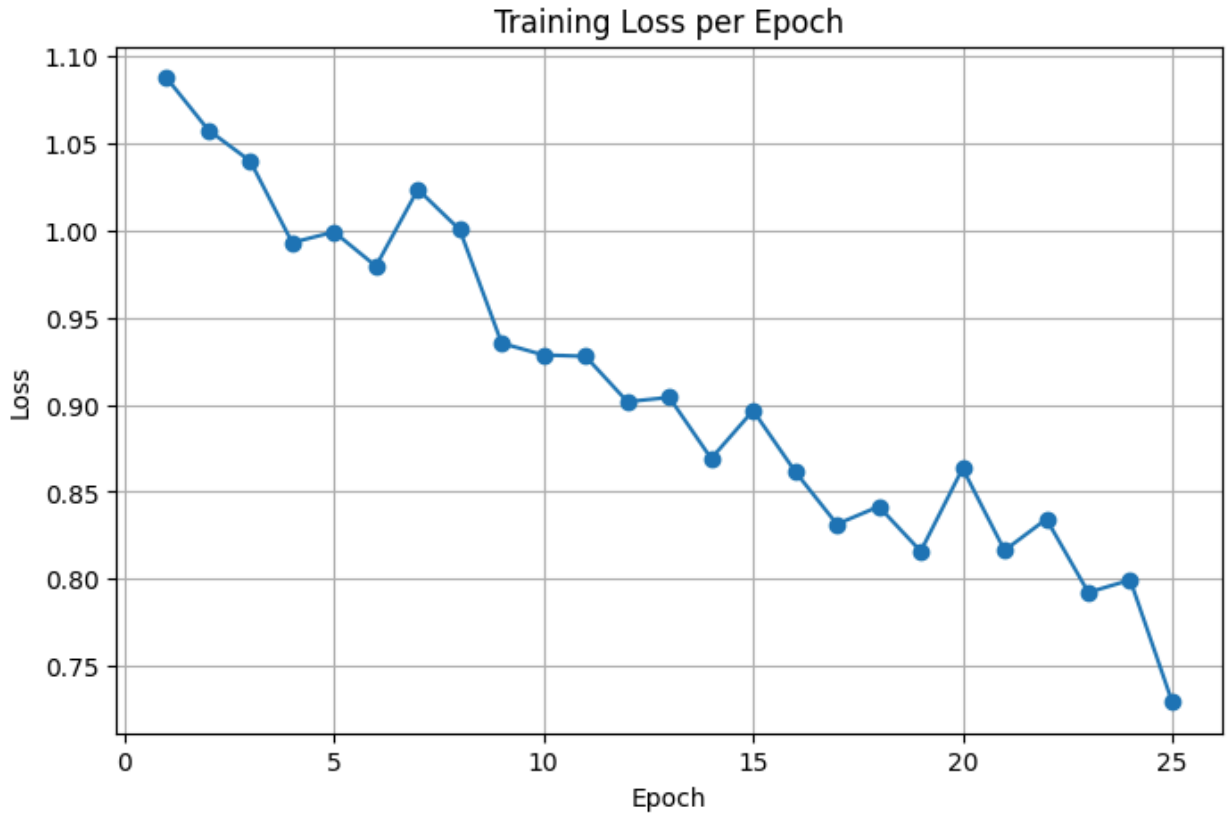
Jessica_Lopez_PDAT_615G_M5.ipynb

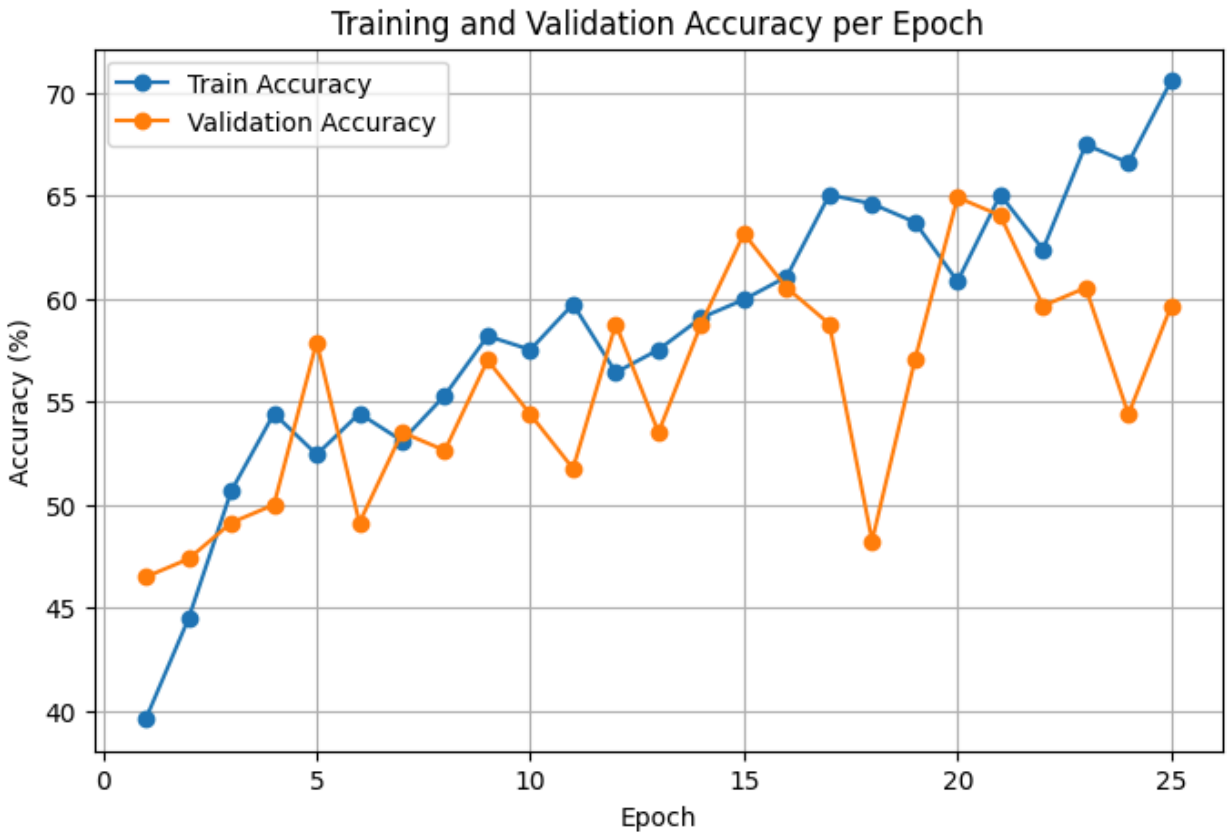
Users > jessicalopez > Desktop > PDAT_615G > Jessica_Lopez_PDAT_615G_M5.ipynb > import os

Generate | Code | Markdown | Run All | Restart | Clear All Outputs | Jupyter Variables | Outline | ...

[23] ✓ 25m 46.5s

```
...  Number of training images: 566
     Number of test images: 566
     Train split: 452
     Validation split: 114
     Epoch [1/25] Loss: 1.0878 | Train Accuracy: 39.60% | Validation Accuracy: 46.49%
     Epoch [2/25] Loss: 1.0578 | Train Accuracy: 44.47% | Validation Accuracy: 47.37%
     Epoch [3/25] Loss: 1.0399 | Train Accuracy: 50.66% | Validation Accuracy: 49.12%
     Epoch [4/25] Loss: 0.9931 | Train Accuracy: 54.42% | Validation Accuracy: 50.00%
     Epoch [5/25] Loss: 0.9992 | Train Accuracy: 52.43% | Validation Accuracy: 57.89%
     Epoch [6/25] Loss: 0.9796 | Train Accuracy: 54.42% | Validation Accuracy: 49.12%
     Epoch [7/25] Loss: 1.0237 | Train Accuracy: 53.10% | Validation Accuracy: 53.51%
     Epoch [8/25] Loss: 1.0005 | Train Accuracy: 55.31% | Validation Accuracy: 52.63%
     Epoch [9/25] Loss: 0.9353 | Train Accuracy: 58.19% | Validation Accuracy: 57.02%
     Epoch [10/25] Loss: 0.9286 | Train Accuracy: 57.52% | Validation Accuracy: 54.39%
     Epoch [11/25] Loss: 0.9278 | Train Accuracy: 59.73% | Validation Accuracy: 51.75%
     Epoch [12/25] Loss: 0.9017 | Train Accuracy: 56.42% | Validation Accuracy: 58.77%
     Epoch [13/25] Loss: 0.9042 | Train Accuracy: 57.52% | Validation Accuracy: 53.51%
     Epoch [14/25] Loss: 0.8691 | Train Accuracy: 59.07% | Validation Accuracy: 58.77%
     Epoch [15/25] Loss: 0.8966 | Train Accuracy: 59.96% | Validation Accuracy: 63.16%
     Epoch [16/25] Loss: 0.8617 | Train Accuracy: 61.06% | Validation Accuracy: 60.53%
     Epoch [17/25] Loss: 0.8314 | Train Accuracy: 65.04% | Validation Accuracy: 58.77%
     Epoch [18/25] Loss: 0.8415 | Train Accuracy: 64.60% | Validation Accuracy: 48.25%
     Epoch [19/25] Loss: 0.8158 | Train Accuracy: 63.72% | Validation Accuracy: 57.02%
     Epoch [20/25] Loss: 0.8634 | Train Accuracy: 60.84% | Validation Accuracy: 64.91%
     Epoch [21/25] Loss: 0.8163 | Train Accuracy: 65.04% | Validation Accuracy: 64.04%
     ...
     Epoch [24/25] Loss: 0.7993 | Train Accuracy: 66.59% | Validation Accuracy: 54.39%
     Epoch [25/25] Loss: 0.7293 | Train Accuracy: 70.58% | Validation Accuracy: 59.65%
```


Training Loss per Epoch
Loss
Epoch
1.10
1.05
1.00
0.95
0.90
0.85
0.80
0.75
0
5
10
15
20
25


Training and Validation Accuracy per Epoch
Train Accuracy
Validation Accuracy
Accuracy (%)
Epoch
70
65
60
55
50
45
40
0
5
10
15
20
25